

Microsoft Azure Practical

Microsoft Azure is one of the fastest-growing cloud service providers around, with 95% of Fortune 500 companies trusting the platform with their business. That's why companies across industries are looking to hire developers with Azure expertise, as Azure-related job growth is projected to hit almost 40% over the next decade.

How will you benefit from this Practicals?

You'll learn to:

- Assess the value, cost, and benefits of hosting their applications and systems in the cloud.
- Interpret the benefits/challenges of IaaS vs. PaaS vs. SaaS and where Microsoft Azure fits into each of these.
- Navigate the Azure Portal.
- Choose the correct Azure Compute Service for their applications and storage needs.
- Create a Subscription and Resource Group.
- Manage and clean up Resource Groups
- Create a Linux VM.
- Create an App Service and App Service Plan.
- Manage basic app service settings.
- Justify which resource to use for app deployment based on costs, scalability, availability and workflow.
- Create a SQL Database in Azure.

- Make updates to the schema of an Azure SQL Database and add data to it.
- Create an Azure Blob Storage Container.
- Upload blobs to the container
- Set up configurations for an app service to connect to both a SQL Database and Azure Blob Storage Container
- Differentiate security options between different Azure resources
- Compare security responsibilities between the developer and cloud provider for different applications.
- Implement OAuth2 such as Sign in with Microsoft in their apps.
- Utilize Azure Active Directory for single sign-on and multi-factor authentication in their app.
- Monitor and analyze the log output of an app service running in Azure.
- Distinguish the benefits of a microservice architecture versus a monolithic architecture.
- Compare and contrast different services as a viable option for building microservices in Azure.
- Outline the different microservices patterns in Azure.
- Identify various compute options for microservice: Kubernetes Service, Docker, and Function (as a Service).
- Identify the costs of implementing microservices.
- Define Azure Functions as well as common bindings and triggers.

Azure Account Setup

For these practicals, you will need to set up an Azure account. When you set up a new account, you can get access to some services for free, along with a \$200 credit that can be used for the first 30 days. After you've gotten your account set up, be sure to have a look at the following page on managing your costs in Azure.

Steps to set up your Azure account

1. Go to the [Azure home](#) page.
2. Click on the "Start Free" Button.
3. You can use your existing "GitHub" or "Outlook" account—or click on "Create One".
4. Finish your registration. Note that *you will need to enter your Credit Card information.*

Upgrading Your Azure Account After 30 Days

After the 30-day \$200 credit period you must upgrade your Azure account to gain access to free services for a year. Please upgrade your Azure account in order to continue without incurring costs.

When you upgrade your Azure account, you are really upgrading a Subscription. Subscriptions are tied to payment details and are the resource in the Azure Portal you attach to payment information and Resource Groups. You can find [more details about Subscriptions](#) in the Azure Applications Course

How To Upgrade Your Azure Account:

1. Log into the Azure Portal with the account you wish to upgrade.
Choose to upgrade your account directly from the top bar of the Azure portal, or by searching for subscriptions, selecting the subscription and choosing “Upgrade Subscription” at the top of the Subscription screen .
2. In the Upgrade screen, scroll down and **choose the “Basic - Included (free)” support plan**. If you do not see the Basic choice, make sure you scroll down to find it on this screen and select it.
3. Click on the Upgrade button
4. Re-enter your credit card information and follow the phone verification if prompted.
5. You should get an email confirming the account upgrade

After your account is upgraded you will have access to [these free Azure resources and limits](#) for one year that you will use. Happy cloud computing!

References

[Upgrade Your Azure Subscription](#)

Verify Azure Costs

BE CAREFUL - When you create your Azure account you will be provided a \$200.00 credit with no charge, but this could very easily be used if you're not careful. **Please use the Standard_B1s size for your VM**, and you should be able to keep the AppService at the free tier with no problem.

Cost Management

You've set up your Azure account in an earlier course; to keep tabs on your Azure cost, and **to make sure your costs don't exceed what's provided to you in the Azure free tier**, go to Cost Management + Billing, then to Cost Analysis. This shows a cost breakdown of the resources in your subscription. **Be sure to check this each day**, especially if you are using Terraform to create resources. **Also please enable Auto-shutdown for your VM by going to the Azure Portal > your VM > Operations > Auto-shutdown.**

The course is divided into 3 major sections - Azure Compute Services, Storage Options in Azure, and Security, Monitoring and Logging.



Azure Compute Services

Storage

Security and Monitoring Basics

- Compute Resources
- App Service
- Virtual Machine
- Storage Options
- SQL Database
- Blob Storage
- Azure Active Directory
- Microsoft Authentication Library (MSAL)
- Monitoring and Logging in Azure

Introduction to Cloud Development

The **cloud** is a collection of servers on the internet that store and manage data, run apps, and deliver content such as email or videos. **Cloud computing** is the delivery of software and storage over the Internet (i.e. *the cloud*).

Some of the benefits of using cloud computing are:

- Cost
- Scale
- Reliability
- Security

The three major types of cloud services are **Public**, **Private**, and **Hybrid** cloud. The major differences between these are in where they are deployed, and who manages them. Microsoft OneDrive, Microsoft Azure, and Microsoft Office365 are examples of public cloud options.

Cloud Developers perform some of the following:

- Plan and design cloud based apps
- Monitor, maintain and support cloud applications
- Develop workflows and processes

Microsoft Learn Resources

- [Cloud Concepts - Principles of cloud computing](#)
- [Foundations of cloud computing for developers](#)

1969 - JCR Licklider aimed to create a system where information could be accessed anywhere in the world called the “Intergalactic Computer Network”.

1980s - Supercomputing Centers start to form and Commercial Internet Service Providers start to emerge in the late 1980s.

1990 - The internet becomes visible to all when Tim Berners-Lee invents the World Wide Web.

1999 - Salesforce.com launches and becomes a pioneer in delivering enterprise applications over the internet, which is known as Software-as-a-Service(SaaS). This paves the way for other SaaS applications such as Microsoft 365 and Gmail.

2002 - Amazon launches its Amazon Web Services (AWS) platform. AWS was formally launched as a business unit in 2006.

2006 - Amazon launches its Elastic Compute Cloud (EC2), a commercial web service that allows users to rent computers to run their applications. EC2 paved the way for application delivery over the internet, which allowed web-scaled businesses like Netflix and Spotify to exist.

2007 - Dropbox introduces its file hosting service, and with that cloud storage becomes a commodity.

2008 - Google launches its Google App Engine (GAE) Platform-as-a-Service (PaaS). This service allowed developers to host their web applications on Google's managed data centers.

2010 - Microsoft launches its cloud computing platform, Azure, after announcing it back in 2008. Azure now covers Software-as-a-Service(SaaS), Platform-as-a-Service(PaaS), and Infrastructure-as-a-Service(IaaS).

2011 - IBM launches SmartCloud, a suite of enterprise-class cloud computing technologies for building private, public, and hybrid clouds.

2013 - Google launches Google Compute Engine (GCE) as an addition to its Google Compute Platform. This is an Infrastructure-as-a-Service (IaaS) component of the platform that allows a user to spin up Virtual Machines (VMs) on demand

Azure is a public computing platform with tremendous value and benefits:

- Scalability: can dynamically handle changes in volume, bandwidth, storage size.
- Availability: redundant on a global scale with 99.9%+ uptime (see SLA by service [here](#))
- Security: replicated data helps protect against natural disaster, while authentication strategies help secure access to the data
- Standard delivery pipeline development services, such as:
 - Source control
 - Unit testing

- Integration testing
- Delivery
- Live development tools

Azure products span multiple categories, such as Compute, Analytics, Databases, A.I. and Machine Learning.

The Azure products this course will focus on are:

1. App Services
2. Virtual Machines
3. Azure SQL Databases
4. Blob Storage
5. Azure Active Directory

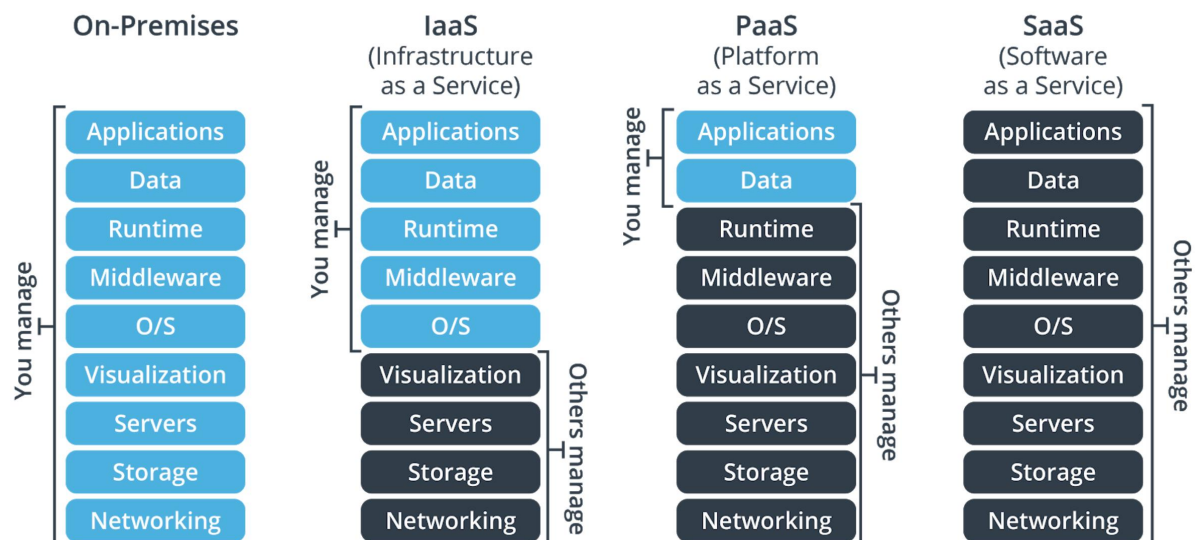
Microsoft Learn Resources

- [Align requirements with cloud types and service models in Azure](#)
- [Core Cloud Services - Azure architecture and service guarantees](#)
- [Core Cloud Services - Manage services with the Azure portal](#)

IaaS, PaaS, and SaaS

In contrast to on-premises solutions, there are three types of services that Azure offers us:

- Infrastructure as a Service (IaaS) - removes the expense of up-front costs of hardware, software and test environments
- Platform as a Service (PaaS) - handles networking, provides middleware and development & database tools
- Software as a Service (SaaS) - provides end-users access to online cloud solutions



Azure services organized by IaaS, PaaS and SaaS

Tools & Environment Checklist

It's time to get all the tools you need to get started with the course!

Task List

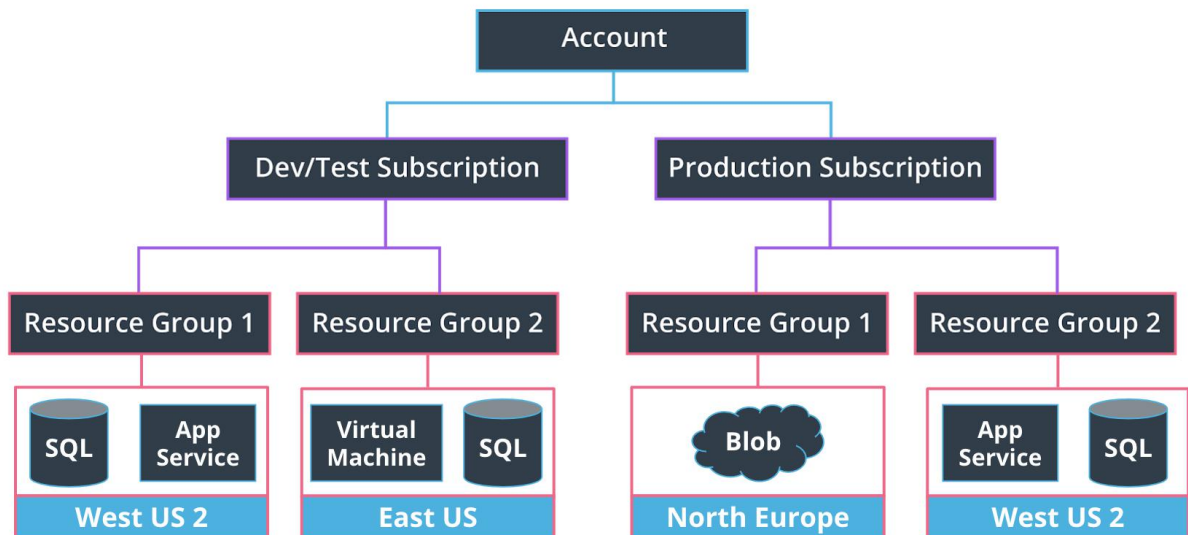
- Sign up for an Azure free account [here](#), if you haven't already.
- Create a Github account [here](#).
- Install Python 3.6 or above [here](#).
- Download and install the Visual Studios Code editor [here](#).
- Install Azure CLI tools for VS Code [here](#), or search for it within VS Code.
- Additionally, install Azure CLI for your terminal [here](#).

Subscriptions and Resource Groups

Azure uses a hierarchical system to keep resources organized and to manage expenses easily.

This starts at the top with your **Azure Account**. The next level down is the **Subscription** level; in this course you will likely only work with one subscription, but it's likely in your day-to-day Azure work that you may be working with more than one. For instance, there may be one subscription for development and testing, and another for production systems.

Below the subscription level is where you'll find **Resource Groups**. These help to organize resources you use, such as Virtual Machines and App Services (as well as storage options and other resources we'll discuss in later lessons), in order to make resource management easier. You may have a resource group for a specific project, or because resource groups are tied to a **Region**, you may have resource groups containing similar resources in multiple locations across the world. A region contains at least one data center, but could have multiple data centers that are close by and networked together through a low-latency network. There are over 60 regions available worldwide and available in 140 countries, such as East US and Japan West.



Resource groups can encompass multiple services in a region.

When choosing a region, it's important to consider what you are trying to achieve. For development and testing purposes, you likely want a region close to yourself; for production purposes, you often want resources to be close to your user. Keep the following in mind:

- Service availability - Some services may not be available in a particular region.
- Performance - Latency determines network service performance; are you creating resources for yourself or your end user?
- Cost - Costs of services vary by region. If latency isn't an issue, you might want to deploy your services in the cheapest region.

Creating a Resource Group in the Azure Portal

To set up a resource group in the Azure Portal:

- Go to the [Azure Portal homepage](#)
- Click "Create a Resource Group"
- Search for "Resource group" and click "Create"
- You'll need to select the subscription for the resource group, name it, and select a region
- Make sure once you select "Review and create" to actually review the information! While it doesn't take too much time to go back and create a new resource group, you'll want to check for any typos in the name or if the wrong region was selected. When we work out of Azure CLI later, this becomes even more important, as you could cause other scripts to fail if things are named incorrectly or are in an inappropriate region.
- Click "Create" once you are done reviewing.

You'll practice this yourself in the upcoming exercise.

Microsoft Learn Resources

- [Manage resources in Azure](#)
- [Move Azure resources to another resource group](#)
- [Control and organize Azure resources with Azure Resource Manager](#)

Alternative Solution: Subscriptions and Resource Groups

There is an alternative way to create a resource group using Azure CLI. Use whichever method you feel more comfortable with moving forward.

Steps to Create a Resource Group using the Azure CLI:

1. Log in using `az login`.
2. We'll use the Azure CLI command `az group create` and pass in two flags:
 - `resource group --name`
 - `--location` which is the same as the "region" field on the Azure portal.
3. If you want to see a list of all locations, you can run `az account list-locations -o table` to output the list in table format.
4. I want to create my resource group in the West US 2 region:

```
az group create --name resource-group-west --location westus2
```

The resource group will be created and a JSON response will be returned with the status.

Alternatively, you can check out the screenshot below for how you might do the same from the Portal (note that the below uses the West US region and not West US 2).

Microsoft Azure

Search resources, services, and docs (G+J)

Home > Resource groups > Create a resource group

Create a resource group

BasicsTagsReview + create

Resource group - A container that holds related resources for an Azure solution. The resource group can include all the resources for the solution, or only those resources that you want to manage as a group. You decide how you want to allocate resources to resource groups based on what makes the most sense for your organization. [Learn more](#)

Project details

Subscription * ⓘ

Azure subscription 1

Resource group * ⓘ

resource-group-west

Resource details

Region * ⓘ

(US) West US

Review + create

< Previous

Next : Tags >

Creating a resource group named `resource-group-west`

Azure CLI Documentation

You can reference the [Azure CLI documentation](#) for further information on other commands.

Virtual Machines vs. App Services

Virtual Machines

Azure Virtual Machines (VMs) provide infrastructure as a service (IaaS) by allowing you to create and use virtual machines in the cloud.

Some of the benefits of VMs are:

- VMs allow you full access and control of the VM.
- Lower up-front cost compared to purchasing and maintaining hardware.
- Support of both Linux and Windows VMs.
- Multiple types to choose from, such as compute or memory-optimized VMs, along with varying amounts of CPU, RAM and storage.
- VMs allow for the installation of custom images and are an excellent choice for migrating from an on-premises server to the cloud.
- Multiple VMs can be grouped to provide high availability, scalability, and redundancy. There are two options when it comes to scaling—Virtual Machine Scale Sets and Load Balancers. These will be covered in a different course.

Some of the limitations of VMs are:

- They are more expensive
- They can be more time consuming for the developer than other compute options

App Service

Azure App Service is an HTTP-based service for hosting web applications, REST APIs, and mobile back ends. It is a Platform as a Service (PaaS) that allows a developer to focus on the application while Azure takes care of the infrastructure.

Some of the benefits of using an App Service are:

- Support of multiple languages, such as .NET, .NET Core, Java, Ruby, Node.js, PHP, or Python
- High availability, auto-scaling and support of both Linux and Windows environments.
- Continuous deployment model using GitHub, Azure DevOps, or any Git repo.
- Vertical or Horizontal scaling. Vertical scaling increases or decreases resources allocated to our App Service, such as the amount of vCPUs or RAM, by changing the App Service pricing

tier. Horizontal scaling increases or decreases the number of Virtual Machine instances our App Service is running.

- You can set the amount of hardware allocated to host your application, and cost varies based on the plan you choose. There are three different tiers - Dev/Test, Production, and Isolated. We'll be using the free option within Dev/Test for the exercises in this course.

Some of the limitations of an App Service are:

- You have limited access to the host server, so you are unable to control the underlying OS or install software on the server.
- You're always paying for the service plan, even if your services or application isn't running.
- There are hardware limitations, such as a maximum of 14GB of memory and 4 vCPU cores per instance
- While they support multiple languages, as noted in the benefits above, they are limited to just using those languages (as of when this course was built).

Use Cases

Each of these has their own use cases, although sometimes there is still some ambiguity on when to use each. Virtual Machines are usually better when you need control of the underlying operating system or are using custom software to support your needs; an app service is typically better for lightweight applications and services, especially when you don't have the need for high performance compute services. Additionally, you'll need to take into consideration the hardware limitations of App Services, as noted above.

Creating a Virtual Machine

In this exercise, you will create resources (Virtual Machines) in Azure. **Once you create these resources they will result in charges in your Azure account if they are not stopped or deleted.** Make sure you stop or delete running resources immediately after completing this exercise or while you are not actively working on it.

The steps I took to create a Linux Virtual Machine on Azure were:

1. On the homepage, click "Create a resource"

2. Click on "Compute" in the left menu
3. Click on "Virtual Machine"
4. Create a Linux VM with the following details:
 - Subscription: This will vary for you. Should default to your subscription.
 - Resource Group: "hello-world-rg"
 - VM Name: "linux-vm-west"
 - Region: West US 2 or a region closest to you that's available
 - Availability Options: Default option of "No infrastructure redundancy required" is fine here.
 - Image: Ubuntu Server (any version) but I'm going to use the latest version 18.04 LTS
 - Azure Spot instance: No
 - Size: Click on "Select Size" and select "Standard B1ls"
 - Authentication type: Password
 - Username: (any username you choose) something other than the default "AzureUser" I used "udacityadmin"
 - Password: (any password you choose) "Udacityadmin@123"
 - Inbound Port Rules: "Allow Select Ports" and make sure from the drop-down menu, 22 and 80 are selected.

To connect to the created VM, I did the following (note that Windows users should use PowerShell or Bash):

1. We'll need our admin username we set when creating the VM - "udacityadmin"
2. We'll need the public IP address of our VM. You can use the following command to grab the IP addresses for a particular VM from the CLI— `az vm list-ip-addresses -g <RESOURCE-GROUP> -n <VIRTUAL-MACHINE-NAME>` or use Azure portal.
3. Next, we're going to copy a basic Flask app from my local machine to the VM. We'll be using the secure copy utility—`scp -r <SOURCE-DIR> [ADMIN-NAME]@[PUBLIC-IP]:<TARGET-DIR>`. In this example, I used—`scp -r ./web udacityadmin@IPADDRESS:/home/udacityadmin`

NOTE: The first time you try connecting to the VM, you'll see a similar message to the one below and should answer 'yes' to permanently add the IP address to the list of known hosts.

The authenticity of host '52.191.135.139 (52.191.135.139)' can't be established.

ECDSA key fingerprint is SHA256:7bBVTsYNImhXxAn+xscCHm/OkcodHZS6l5VSKO3GP8c.

Are you sure you want to continue connecting (yes/no)?

4. Once the files have been copied to the VM, we can connect to the VM using—`ssh [ADMIN-NAME]@[PUBLIC-IP]`
5. We can use `ls` to see the web directory we just uploaded

6. Python 3 is already installed on the VM. We'll install Python Virtual Environment and NGNIX to use as a reverse proxy `sudo apt-get -y update && sudo apt-get -y install nginx python3-venv`
7. Before we run the app, we have to configure Nginx to redirect all incoming connections on port 80 to our app that is running on localhost port 3000
 - By default, Nginx has a default page that is displayed. If you visit the public IP address in your browser, you should see this page rendered.
 - We'll navigate to the **/etc/nginx/sites-available** directory —
`cd /etc/nginx/sites-available`
 - We'll first unlink the default site using `sudo unlink /etc/nginx/sites-enabled/default`
 - Then we'll create a new file **reverse-proxy.conf** in the **/etc/nginx/sites-available**—`sudo vim reverse-proxy.conf`
 - We're going to add the following code to this file:


```

server {

    listen 80;

    location / {

        proxy_pass http://localhost:3000;

        proxy_http_version 1.1;

        proxy_set_header Upgrade $http_upgrade;

        proxy_set_header Connection keep-alive;

        proxy_set_header Host $host;

        proxy_cache_bypass $http_upgrade;

    }

}

```

8. Now we'll activate the directories by creating a sym link to the

/sites-enabled directory `sudo ln -s /etc/nginx/sites-`

`available/reverse-proxy.conf`

`/etc/nginx/sites-enabled/reverse-proxy.conf`

- We have to restart nginx so the changes take effect. `sudo service nginx restart`

9. cd to `web`

10. Create venv `python3 -m venv venv`
11. Activate the env `source venv/bin/activate`
12. Upgrade pip in our virtual environment and then Install dependencies `pip install --upgrade pip`
`pip install -r requirements.txt`
13. We'll run our app `python application.py`
14. In a web browser, we can visit the public IP address of the VM and you should see the application
15. Type "exit" to disconnect from the VM

Note: Azure free account only allows 750 hours of free hosting. Make sure to delete this VM after creation for any exercises to avoid charges.

Cleanup

If we no longer need a resource, we can delete them through the portal.

1. From the homepage, click on "Resource Group"
2. Click on the resource group you want to manage

You have two options—"Delete resource group" or if you want to keep the resource group, you can click on the individual or collection of resources you want to delete and click on "Delete"

Creating an App Service

I took the following steps to create an App Service Web App using the Azure Portal

- 1. On the homepage, click "Create a resource"**
- 2. Search for "Web App"**
- 3. Click "Create"**
- 4. Select your subscription**
- 5. Select your resource group "resource-group-west"**
- 6. Enter a name for your web app—This needs to be a unique name and is unique to Azure as a whole and not just your Azure account. I used "hello-world1234" as my unique name.**
- 7. For Publish, select "Code"**
- 8. For the "Runtime Stack", select "Python 3.7" or greater.**
- 9. For the "Operating System" select "Linux"**
- 10. Select a region for your app service**
- 11. Create a new App Service Plan. You can keep the default name Azure gives you or you can create your own name.**
- 12. For SKU and size, select "F1" (Free).**
- 13. Click "Review + Create"**
- 14. Click "Create"**

Then, I clicked on the URL to see the app service, which confirmed the app service was up and running. Now we need to deploy our code.

Note: Azure free account only allows 1 Linux App Service of size F1. You will need to delete the App Service Web App along with the App Service Plan after each exercise since we will create other Linux App Services throughout this course.

To deploy an App Service from a GitHub repository, I did the following:

- 1. Go to Deployment Center**
- 2. Choose GitHub**
- 3. Choose org and repo**
- 4. Go through the prompts; deployment takes a few minutes**
- 5. Go to URL of web app and should see the app deployed**

Cleanup

If we no longer need a resource, we can delete them through the portal.

- 1. From the homepage, click on "Resource Group".**
- 2. Click on the resource group you want to manage.**
- 3. You have two options—"Delete resource group" or if you want to keep the resource group, you can click on the individual or collection of resources you want to delete and click on "Delete".**

Solution: Creating an App Service

In this solution video, I showed you an alternative way to create and deploy an Azure App Service Web App using Azure CLI. Use whichever method you feel more comfortable with moving forward.

Steps to Create and Deploy an App Service Web App from a Directory using Azure CLI:

1. Sign in to Azure `az login`
2. cd to `web` directory

Run the following command:

```
az webapp up \
```

```
--resource-group resource-group-west \
```

```
--name hello-world1234 \
```

```
--sku F1 \
```

```
--verbose
```

3. If you visit the URL, you should see your site deployed.

If you want to update your app, make changes to your code and then run (Note: this may not update new requirements you may have added):

```
az webapp up \
```

```
--name hello-world1234 \
```

```
--verbose
```

4. Alternatively, you can create the initial web app from the Portal as shown in the below screenshot.

Microsoft Azure

Home > App Services > Web App

Web App

Basics Monitoring Tags Review + create

App Service Web Apps lets you quickly build, deploy, and scale enterprise-grade web, mobile, and API apps running on any platform. Meet rigorous performance, scalability, security and compliance requirements while using a fully managed platform to perform infrastructure maintenance. [Learn more](#)

Project Details

Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * Azure subscription 1

Resource Group * resource-group-west [Create new](#)

Instance Details

Name * linux-app-west .azurewebsites.net

Publish * Code Docker Container

Runtime stack * Python 3.7

Operating System * Linux Windows

Region * West US
 ⓘ Not finding your App Service Plan? Try a different region.

App Service Plan

App Service plan pricing tier determines the location, features, cost and compute resources associated with your app. [Learn more](#)

Linux Plan (West US) * (New) ASP-resourcegroupwest-8e35 [Create new](#)

Sku and size * **Free F1**
 1 GB memory
 [Change size](#)

[Review + create](#) < Previous Next : Monitoring >

An example of creating the App Service in the Portal

Cleanup

If we no longer need a resource, we can delete them through the portal. The quickest way to do this from the CLI is to delete the resource group. This will delete all resources in that group

```
az group delete -n resource-group-west
```

Alternatively, if you want to just delete the App Service and App Service plan individually, you can do so with the following commands:

Delete an App Service

```
az webapp delete \
```

```
--name hello-world1234 \
```

```
--resource-group resource-group-west
```

Delete an App Service plan

```
az appservice plan delete \
```

```
--name [App Service Plan Name] \
```

```
--resource-group resource-group-west
```


Big Picture: Azure Storage

Some of the benefits of using Azure for storage are:

- **Automated backup and recovery**
- **An option to replicate data at multiple data centers worldwide to help prevent outages from unplanned events, such as hardware failure.**
- **Data analytics support**
- **Data encryption for added security**
- **Support for the storage of multiple data types. Azure is designed to hold 3 main types data—relational data, non-relational data or NoSQL data, and unstructured data such as images.**
- **Scale up or scale out when demand is high and scale back when demand is low.**
- **Eliminate the expense of having to purchase, install, configure, and maintain on premises hardware.**

Azure Storage Options

Among others, some of the Azure Storage options are:

- **Azure SQL Server and SQL Database**
- **Azure Blob Storage**

- **Azure CosmosDB**
- **Disk Storage**
- **Azure Data Lake Storage**
- **HPC Cache**

We could spend an entire course just on Azure storage, so we'll focus on two of these services in this lesson—Azure SQL Database and Azure Blob Storage. CosmosDB will be covered in another course.

We also covered the three tiers of blob storage - hot, cool and archive. We'll come back to this later in the section on Azure Blob Storage.

Azure SQL Databases

- **Azure SQL databases are used for structured, relational data.**
- **In Azure, you also need to first create a related SQL server to hold the database, although you can create multiple SQL databases under a single SQL server.**
- **Azure SQL databases do not have a free tier option like we saw with App Services earlier; however, when you sign up for a new free Azure account, you get 12 months of up to 250GB of Azure SQL database storage. The lowest tier is currently ~\$5 / month otherwise for a small amount of storage.**

Creating an Azure SQL Database in the Portal

To create an Azure SQL Database in the Azure Portal:

- 1. Find the "SQL databases" service in Azure.**
- 2. Click "Create SQL Database".**
- 3. Select the appropriate subscription and resource group (likely "resource-group-west").**
- 4. Enter a database name.**
- 5. If you already have a SQL server, you can use it; however, you likely need to click "Create New".**
 - If creating a new SQL server, enter a server name, and then admin and password - make sure you can remember these, or you will not be able to access the server when necessary. The admin name cannot just be "admin".**
 - Set the location to match the resource group.**
- 6. Keep SQL elastic pool on the default of "No".**
- 7. Under Compute+Storage, click on "Configure Database".**

While General Purpose won't end up charging you anything for the short time the server is live for these exercises, you might as well press "Looking for basic, standard, premium?", and change it to "Basic".
- 8. Press the "Next: Networking" button, then select "Public Endpoint", and set both of the Firewall rules that appear to**

"Yes". This will allow us to more easily access the SQL database later on within our app.

- 9. Click "Review + Create" and then "Create" to create the database, then wait for it to deploy.**

Adding Data to the Database

To add data to the SQL Database, I performed the following:

- 1. Once the SQL database is deployed, click on its name to access it (you may need to go back to the main "SQL databases" page in Azure).**
- 2. Click on the "Query editor" in the left side menu, and then log in with my SQL Server credentials.**
- 3. I pasted the query from my `posts-table-init.sql` script into the query window, and hit "Run"; you should see `Query succeeded: Affected rows: 3`.**
- 4. On the left side of the query window, I clicked on the "Tables" folder, and double-checked that the `dbo.POSTS` table was successfully created with the correct fields.**
- 5. I ran a new query with `SELECT * FROM posts`, to see the three posts listed.**

You'll get to practice this yourself in the upcoming exercise.

EXERCISE:

In this exercise, you will create an Azure SQL Database and add data to it. This is an [Azure account first-year free service](#). In a later exercise, you will connect an app to the database, so that you can both get and post data from the app into the database.

1. Deploy a SQL Database in Azure called “database-west” to the “resource-group-west” resource group (or whichever resource group you have been using in earlier exercises).
2. Within the Azure portal, add the data from the script in the `sql_scripts` directory here to the database. You should be able to run a `SELECT` query afterward on the `animals` table to see the example animals populated in your database.

Note: Azure free accounts only allow 250GB of free storage using SQL Databases. Once you get to the end of this lesson, you will want to delete the SQL Database and SQL Server to avoid incurring any charges, as you'll create a different SQL Database in the final course project.

Supporting Materials

[sql-scripts.zip](#)

Blob Storage

Before discussing Blob Storage Accounts, you may need to know what a Blob is in the first place.

A Binary Large Object or Blob is a data type that can store unstructured (binary) data and is used to store things like images or videos in a database. Blobs are not the go-to choice when it comes to structured data that is queried frequently, such as user profile information. They have higher latency than memory and local disk and don't have the indexing features that make databases efficient at running queries. Blobs are commonly used in combination with databases to store non-queryable data, such as a profile picture for a user's profile. Each user record in the database would include the URL of the blob containing the user's photo.

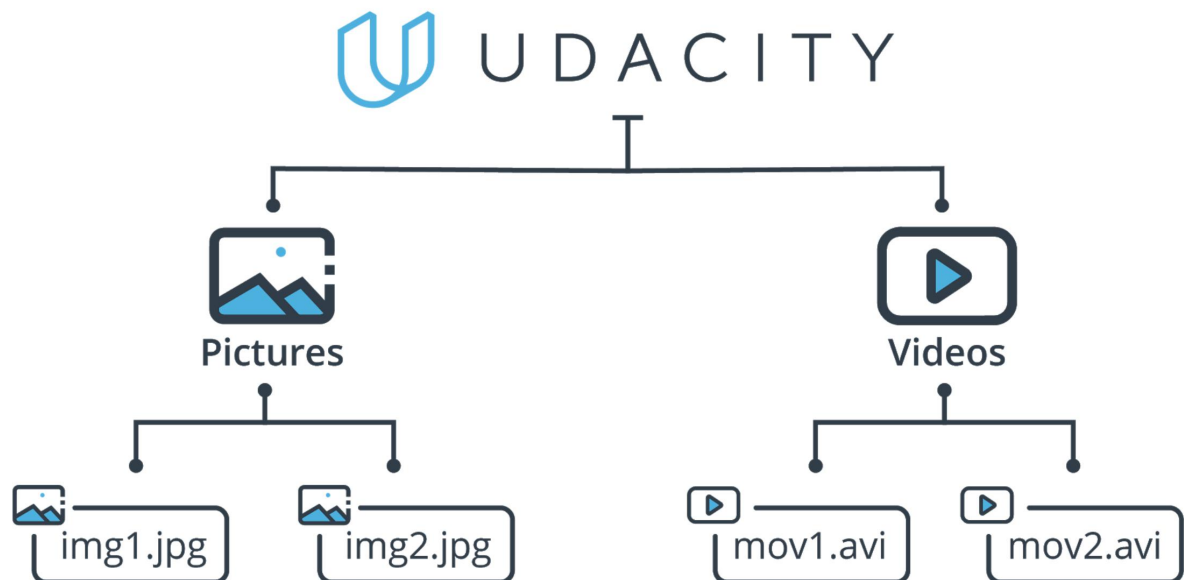
Azure Storage Accounts

An Azure storage account can store data objects you create, such as blobs and files. This storage account provides a unique namespace in Azure for your data, and every item that you store has an address that includes your unique account name. It's worth noting the name of the storage account can contain only lowercase letters and numbers.

In this practical, you'll mostly utilize *General-purpose v2* storage accounts, which provide support and the latest features for Azure Storage services such as blobs, Data Lake Gen2, Files, Disks, Queues,

and Tables. This type of storage account is recommended for most scenarios when using Azure Storage services.

Storage accounts can contain multiple blob containers within them, such as "images" and "movies" containers, to organize different data files. From there, each container can have many blobs inside of them (the files themselves).



The Blob Storage Hierarchy - "Udacity" is the storage account, with "Pictures" and "Videos" as containers, each with more than one blob inside.

Blob Storage Lifecycle.

Blob storage offers a rule-based policy you can use to transition your data between these tiers to optimize performance and cost. This is shown again in the below graphic - while your blobs may start in a Hot container, they could be moved to a Cool container if they have not been modified in the past 30 days. From there, if they have not been modified in 90 days, they could be transitioned to Archive storage; you could then perhaps delete them after a year of non-usage.



The blob storage lifecycle

Creating Blob Storage in the Portal

To create a blob storage account on Azure Portal, I did the following:

Create a Storage Account in Azure:

1. Click Create a resource
2. Search for storage account
3. Select the appropriate subscription and resource group (likely “resource-group-west”).
4. Enter a storage account name, note—it can only be lowercase letters and numbers. “helloworld1234”
5. Set the location to West US 2 to match the resource group.
6. Leave "Performance" as Standard, while making sure "Account kind" is StorageV2. This could also be BlobStorage, but StorageV2 is preferred.
7. We'll leave replication on the default setting
8. Set "Access tier" to Cool - in a live production app, you may need this to be Hot, but in our case, Cool will work just fine.
9. Click "Next: Networking" to confirm that the storage account is using a Public endpoint (all networks)
10. Click "Next: Data Protection", "Next: Advanced", then set "Blob public access" to "Enabled".
11. Click "Review + Create" and then "Create" to create the database, then wait for it to deploy.

To add a Blob Container called `images`:

1. Once the Storage account is deployed, click on its name to access it (you may need to go back to the main "Storage accounts" page in Azure).
2. Click on the "Containers" button in the storage account's "Overview" page.
3. Click "+ Container", then add the name of `images`.
4. Set "Public access level" to Container, and click "Create".

NOTE: I had to refresh my page a few times before I could access the container.

Adding Images to the Container

To add an image to the Blob Container:

1. Click on the container named `images`.
2. Click "Upload", and add an image of your choosing.
3. Once the image appears within the container view, click on it, and copy the "URL" property.
4. Paste the URL into your browser to see if the image appropriately loads.

Connecting Your App to Storage

To connect an app to the storage we've set up, we need a few things from each storage service.

From the SQL server and database:

1. SQL Server server name (the name of the sql server with .database.windows.net appended to it)
2. Admin username
3. Admin password
4. SQL Database name

From blob storage:

1. Storage account name
2. A storage account access key
3. Container name

In our case, we're keeping the management of these values a bit simpler with a `config.py` file that will be imported into the primary app file.

Azure Storage Blob Library for Python

In order to interact with our Azure blob storage from within the Python web app, we'll need the [Azure Storage Blob Library](#). Note that you can install this with `pip install azure-storage-blob`, and you'll

need to make sure to include the library in your `requirements.txt` file in your own apps.

We will largely focus on the `BlobServiceClient` class. This class has three methods we'll use:

- `get_blob_client` - creates a blob client using the filename as the name for the blob
- `upload_blob` - upload the file to the blob container
- `delete_blob` - delete a blob from a blob container

Uploading a blob

Here is the code I included for uploading a blob.

```
from azure.storage.blob import
BlobServiceClient

blob_container = app.config['BLOB_CONTAINER']
storage_url =
"https://{ }.blob.core.windows.net/".format(app.
config['BLOB_ACCOUNT'])
blob_service =
BlobServiceClient(account_url=storage_url,
credential=app.config['BLOB_STORAGE_KEY'])
```

```
blob_client =  
blob_service.get_blob_client(container=blob_co  
ntainer, blob=filename)  
blob_client.upload_blob(file)
```

Note that I am getting the `BLOB_CONTAINER` name from the configuration file, which was attached to the Flask app separately. I do a similar procedure for getting the `BLOB_ACCOUNT` AND `BLOB_STORAGE_KEY` where needed. I then use the aforementioned `get_blob_client` and `upload_blob` functions to upload a file.

You might notice that there is a `filename` and `file` used here - these are not the same things! The `filename` is just the name of the file, e.g. `test_image.png`, while the `file` is the actual file object. So, the blob client is first called to create a blob with the given `filename`, and then the `file` is uploaded into that `filename` blob.

Deleting a blob

Assuming you've already defined the `blob_service` from before, you just need to get a new blob client with `get_blob_client` for the related container and `filename`, then `delete_blob`.

```
blob_client =  
blob_service.get_blob_client(container=container, blob=filename)  
blob_client.delete_blob()
```

It's important to note here you don't need a file to feed to `delete_blob` - by specifying the `filename` when you `get_blob_client`, it already knows what blob you are referring to.

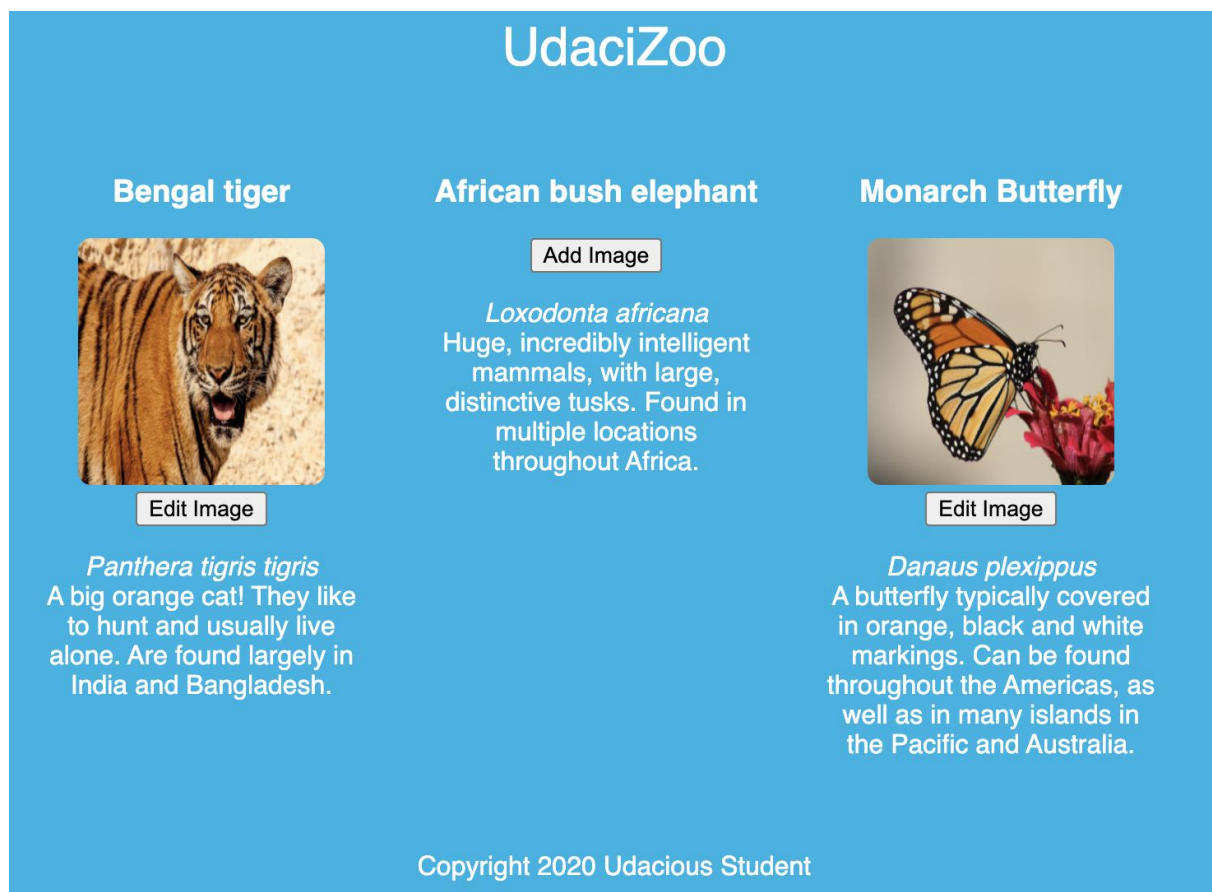
Exercise: Connecting Your App to Storage

In this exercise, you'll be given an application for a virtual zoo, and asked to add connections to your earlier deployed SQL database and Blob Storage to populate the zoo application.

If you already closed down your database and blob storage, you'll want to go back and re-create them before getting started.

Download the starter code for the app here.

1. First, familiarize yourself at least with the `views.py` file in the `FlaskExercise` directly, although you may also want to check out the other files. You can see an example screenshot of the deployed application below.



An example of the connected app

2. Next, add the necessary environment variables to connect to the SQL database in `config.py`.
3. Then, add the necessary environment variables to connect to the Blob storage container in `config.py`.
4. Add the necessary code in `models.py` to work with the `BlobServiceClient` to upload new images and delete any images that are replaced.
5. Run the app on your local machine, and check that the animals are correctly populated from the SQL database.
6. Add some images for each animal. You should be able to check back in your blob container and see that new images were added, and they should populate back to the main page.

While it's not a required part of this exercise, you can also try to deploy the app using either an app service or virtual machine; you shouldn't need any additional changes specific to the storage connections, but other changes may be necessary.

Troubleshooting

Mac users may need to install `unixodbc` as well as related drivers as shown below:

```
brew install unixodbc
```


- Check [here](#) to add SQL Server drivers for Mac. [This StackOverflow post](#) may also help resolve certain issues.

Supporting Materials

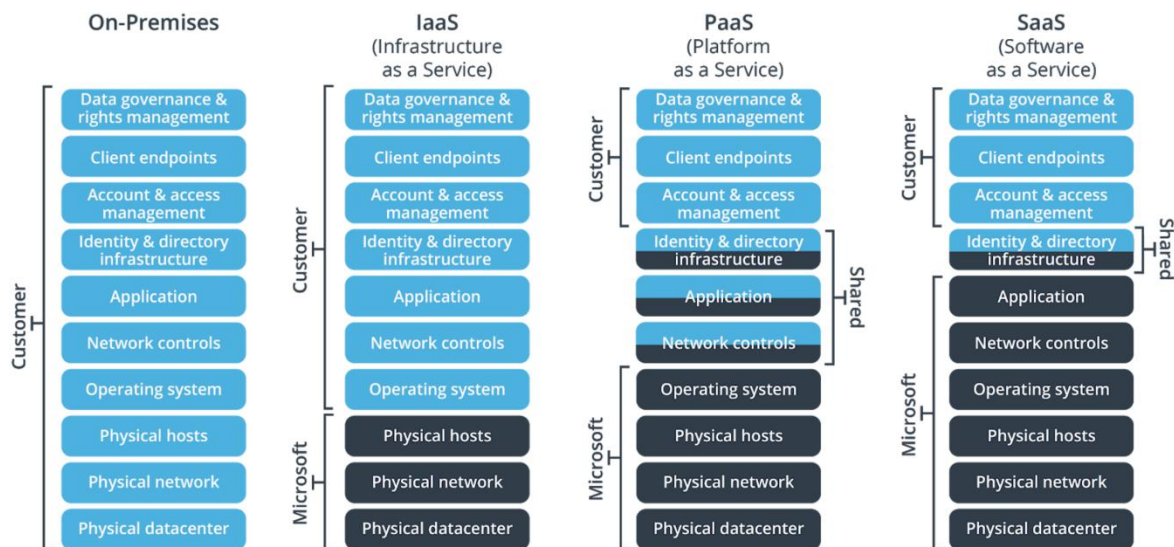
[connect-app-to-storage-starter.zip](#)

Security Responsibilities

Just like we saw early on in the course with technology stack differences between on-premises, IaaS, PaaS and SaaS, security responsibilities also differ among these service types. For the cloud developer, on-premises still puts all security responsibilities onto them. With IaaS, the security of physical hardware shifts to the cloud provider. With PaaS, the operating system security shifts to be the responsibility of the cloud provider, while network, application and identity security are a shared responsibility. With SaaS, the network and application shift responsibility to the cloud provider, but identity stays as a shared responsibility.

It's important to note here that some security responsibilities *always* stay with the cloud developer:

- Account and access management (E.g. do you give the appropriate people the right access, and revoke access when appropriate?)
- Client endpoints (E.g. do you secure the appropriate endpoints to confidential information?)
- Data Governance and Rights Management (E.g. do you prevent employees from emailing confidential documents to third parties?)



Developer vs. Provider Responsibilities in Azure

Security Options in Azure

There are a ton of services and topics to consider in Azure, each with their own respective use cases. While we will focus on Azure Active Directory, OAuth 2.0 with MSAL, and some aspects of Azure Monitor in this lesson, it's still worth noting the basics of the other services and topics.

- **Azure Active Directory:** Provides single sign-on (SSO) and multi-factor authentication (MFA) capabilities, such as Sign in with Microsoft
- **App Configuration:** Stores application settings in one secure location
- **Key Vault API:** Stores application keys and secrets in one secure location

- **Managed Identities:** A part of Azure Active Directory; This helps streamline providing an app or app user access to other Azure resources
- **Shared Access Signatures:** Give external parties certain limited access (determined by you) to different Azure resources
- **Role-Based Access Controls (RBAC):** Help internally manage who has access to what resources, and what they can do to said resources
- **Azure Monitor:** Provides a wide range of monitoring services such as log analytics, metrics, alerts, and much more
- **Application Insights:** Part of Azure Monitor; This helps monitor performance and other key metrics

Azure Active Directory

Azure Active Directory is Microsoft's solution for single sign-on (SSO) and multi-factor authentication (MFA). We'll be using it in combination with the Microsoft Authentication Library (MSAL) to use "Sign in with Microsoft" buttons in an app, although it can be used more broadly for identity management purposes within an organization. The "tenant" in Azure AD is usually equivalent to an organization.

As you get started with Azure AD, there may be a limit to your access if you are using an enterprise account - for instance, while building this course, I was actually locked out of the main Azure AD tenant for Udacity (for security reasons), and had to use a separate one for building course materials.

Using Azure Active Directory in the Portal

As usual, you can start things off in the Azure Portal by just searching for "Active Directory". From there, you should already be a tenant by default, although you can create a new one, if needed, to keep things separate from your main tenant. Again, if you are on an enterprise account that limits your access here, you may need to use another (personal) account.

To register an app in Azure AD (which does not actually need a deployed app for these steps):

- Click on app registrations on the left panel.
- Enter a user-facing display name.
- I allowed the widest set of accounts to access it (Accounts in any organizational directory and personal Microsoft accounts).
 - When you build your own apps, you may want to be more restrictive, but in this case, it will make it easier for you to allow other users to authenticate with your app. An internal application, for instance, might be limited to a single tenant.

- You can ignore the redirect URI here for now, and revisit that when we get to implementing OAuth 2.0. However, if you already know this for your app, you can enter one here now, and any others necessary later on.
- Once the Application has been registered, you need to copy the Application (client) ID, as it'll be used later in another exercise.
- Next, under manage on the left here, click on certificates and secrets
- Click on "+ New client secret", then enter a description (you can decide on your own desired expiration time, I chose one year). Copy down the string under "Value", and make sure you store it somewhere safe - you won't be able to see it again, and would otherwise have to create a new client secret once again. We'll use this value and the application client ID when implementing OAuth 2.0.

You'll practice this yourself in the upcoming exercise